Ingeniería de Sistemas de Detección Probabilística de Trampas en Counter-Strike 2

El ecosistema competitivo de Counter-Strike 2 (CS2) se enfrenta a una crisis de integridad sin precedentes debido a la sofisticación de los sistemas de asistencia externa. La detección tradicional basada en firmas (VAC) ha demostrado limitaciones frente a trampas de nivel de kernel y dispositivos de hardware externos. En este contexto, la ingeniería de sistemas anti-cheat debe evolucionar hacia el análisis de comportamiento basado en datos públicos y telemetría de red. Este informe detalla el diseño técnico de una plataforma escalable capaz de estimar la probabilidad de fraude mediante el procesamiento de metadatos de partidas, perfiles de usuario y señales de reputación externa. El sistema propuesto no solo busca identificar valores atípicos en el rendimiento, sino también modelar la "huella digital" del comportamiento humano frente a la ejecución robótica de acciones en el juego.[1]

Ingesta de Datos y Protocolos de Integración

La base de cualquier sistema de análisis predictivo es la calidad y profundidad de sus datos. En el caso de CS2, no existe una fuente única que proporcione la totalidad de la información necesaria. Por lo tanto, el sistema implementa una arquitectura de ingesta distribuida que consume datos de proveedores oficiales y de terceros, normalizando la información en un esquema unificado de perfiles de jugador.[3]

Protocolos de la Steam Web API

La Steam Web API actúa como el primer vector de identidad. Se utiliza para establecer la antigüedad de la cuenta, la inversión económica (a través de juegos poseídos) y el historial de sanciones previas. La autenticación se realiza mediante una clave de API obtenida en el portal de desarrolladores de Valve.[5]

Endpoint	Parámetros Clave	Datos Obtenidos	Relevancia Técnica
ISteamUser/GetPlayerSummaries/v2	steamids	timecreated, communityvisibility state, profilestate	Identifica cuentas "quemables" (burner accounts) con poca antigüedad. [5]

ISteamUser/GetPlayerBans/v1	steamids	VACBanned, NumberOfVACBans, DaysSinceLastBan	Proporciona el historial de integridad del usuario. [5]
IPlayerService/GetOwnedGames/v1	steamid, include_appinfo	playtime_forever (AppID 730), playtime_2weeks	Calcula la intensidad de juego y detecta smurfs de alta habilidad y pocas horas. [5]

El sistema debe gestionar los límites de tasa (rate limits) de Valve, que suelen estar en el rango de 100,000 solicitudes diarias. Se implementa un sistema de gestión de claves y rotación de proxies para garantizar la disponibilidad en entornos de alta demanda.[7]

Integración con Faceit Data API

Faceit representa el estándar de oro para el juego competitivo con anti-cheat de kernel. El acceso a su API permite contrastar el rendimiento del jugador en un entorno altamente regulado frente a los servidores oficiales de Valve.[9]

- **Endpoint de Búsqueda de Jugador**: GET /players?game=cs2&external_id={steamid64}. Este llamado traduce el SteamID64 a un FaceitID único.
- **Estadísticas de Rendimiento**: GET /players/{player_id}/stats/cs2. Devuelve campos críticos como el ELO actual, nivel (1-10), win rate y el K/D ratio histórico.[11]
- **Historial de Partidas**: GET /players/{player_id}/history. Permite rastrear la progresión del jugador y detectar aumentos repentinos de habilidad que coincidan con periodos de inactividad.[11]

Agregación de Datos Avanzados: Leetify y CS2.space

Leetify utiliza una solución de raycasting propietaria para analizar archivos de demostración (demos) y extraer métricas de puntería y posicionamiento que no son visibles en las estadísticas básicas.[13] Aunque su API pública es restringida, el uso de agregadores como CS2.space permite obtener un resumen de estas métricas avanzadas mediante una única llamada API: GET /api/profile/{steamId}.[1]

Métrica Leetify	Descripción Técnica	Umbral de Sospecha
Aim Rating	Calificación de precisión basada en micro-ajustes y tiempo de reacción.	> 95 (Excede niveles profesionales promedio). [14]

Positioning Rating	Eficiencia de los ángulos de combate y uso de coberturas.	Muy bajo en relación con un Aim Rating extremadamente alto.
Time to Damage (TTD)	Milisegundos entre la visibilidad del enemigo y el impacto del primer disparo.	< 200ms de forma consistente. [16]

Estrategias de Obtención de Partidas Recientes

Uno de los mayores obstáculos técnicos es que Valve no expone el historial de partidas de CS2 a través de su API web estándar para usuarios generales. El sistema debe emplear métodos alternativos para reconstruir el historial de las últimas 100 partidas.[4]

Sistema de Match Sharing Codes y Game Authentication

El método oficial requiere que el usuario proporcione un "Game Authentication Code" y el código de su última partida. El sistema utiliza el endpoint ICSGOPlayers_730/GetNextMatchSharingCode/v1 para iterar de forma recursiva hacia atrás en el historial del jugador.[17]

Este proceso genera un flujo de datos estructurado:

1. El backend solicita el steamid, steamidkey (auth code) y un knowncode inicial.
2. La API responde con el nextcode si existe una partida posterior disponible.[17]
3. Este ciclo se repite hasta alcanzar el límite deseado (ej. 100 partidas) o agotar el historial disponible.

Alternativas de Scraping y Agregadores de Terceros

Dada la fricción que supone pedir códigos de autenticación a los usuarios, el sistema implementa un "Scraper de Respaldo" que consulta servicios como CSStats.gg o Scope.gg. Estos servicios mantienen sus propios bots de recolección de demos y exponen interfaces web que pueden ser procesadas para extraer el scoreboard de las partidas.[18]

Los campos obligatorios a recolectar por cada partida son:

- **Resultados de Combate**: Kills, Deaths, Assists, Headshot %.[20]
- **Impacto en Ronda**: ADR (Average Damage per Round), MVPs, Utility Damage.[1]
- **Contexto de la Partida**: Resultado (Victoria/Derrota), Score final (ej. 13-11), Mapa jugado y Rango promedio del lobby.[12]

Ingeniería de Características para la Detección de

Anomalías

La simple observación de promedios es insuficiente para detectar tramposos sofisticados ("closet cheaters") que intentan ocultar su ventaja. La ingeniería de características (feature engineering) se centra en la varianza y en métricas de comportamiento imposibles de replicar de forma humana.[22]

Métricas de Variabilidad y Consistencia

Un jugador profesional muestra una alta habilidad, pero también una variabilidad humana natural condicionada por la fatiga, el estado mental y la calidad de los oponentes. Un cheater suele presentar una consistencia estadística artificial.[15]

Se calculan las siguientes variables:

- **Desviación Estándar de K/D**: Un valor extremadamente bajo (consistencia perfecta) o extremadamente alto (comportamiento de "toggle" donde se activa la trampa solo en rondas críticas) es indicativo de fraude.[15]
- **Z-Score de Headshot %**: Se calcula la distancia en desviaciones estándar respecto a la media de la población para el rango específico del jugador. Un $Z > 3.0$ en un rango bajo es una señal de alerta crítica.[25]
- **Skill Jump Detection**: Se mide la pendiente de mejora de ELO o Rating en ventanas de 7 días. Mejoras superiores al percentil 99 de la población sugieren la adquisición de software externo o un servicio de "boosting".[2]

Análisis de Comportamiento Táctico

A partir de los datos de Leetify y servicios similares, se derivan métricas que cuantifican la ventaja de información (wallhacks) y la asistencia de puntería (aimbots).[13]

- **Error de Ángulo de Pre-aim**: Mide cuántos grados debe corregir el jugador su mira una vez que el enemigo es visible. Un error promedio cercano a cero a través de paredes sugiere el uso de ayudas visuales.[13]
- **Probabilidad de Prefire**: Frecuencia con la que el jugador dispara antes de que la telemetría del servidor registre la visibilidad del enemigo. Se correlaciona con el ping para descartar ventajas de red legítimas.
- **Clutch Performance Efficiency**: Ratio de victorias en situaciones de 1vX. Los cheaters a menudo "disimulan" durante la ronda pero aseguran las situaciones de clutch para ganar la partida.[15]

Metodologías de Detección y Clasificación

El sistema no depende de un único juicio, sino que combina tres enfoques analíticos para generar una probabilidad final robusta.[25]

Enfoque A: Sistema Basado en Reglas (Heurísticas)

Este nivel identifica a los "Rage Cheaters" o tramposos flagrantes mediante reglas deterministas de exclusión.

ID Regla	Lógica de Activación	Acción
HS_EXTREME	HS% > 80% en las últimas 10 partidas AND K/D > 2.5	Flag: Alto Riesgo [14]
TTD_IMPOSSIBLE	Mediana de TTD < 180ms en una muestra de 50 bajas	Flag: Sospechoso [16]
ACCOUNT_HYGIENE	Cuenta < 100 horas AND Rango > 20,000 Premier	Flag: Smurf/Cheater [1]

Enfoque B: Análisis Estadístico Comparativo

Se utiliza una base de datos de referencia que contiene las métricas de miles de jugadores, desde niveles plata hasta profesionales. Se posiciona al usuario en esta distribución mediante percentiles. Si un jugador se encuentra en el percentil 99.9 de precisión pero en el percentil 10 de utilidad (granadas) y movimiento, existe una asimetría de habilidades que es característica del fraude.[2]

Enfoque C: Machine Learning No Supervisado (Isolation Forest)

Para detectar patrones complejos que escapan a las reglas manuales, se implementa un modelo de **Isolation Forest**. Este algoritmo es particularmente efectivo porque no requiere un dataset etiquetado (cheater vs no-cheater), sino que identifica anomalías basándose en la facilidad con la que un punto de datos puede ser aislado del resto de la muestra.[25]

- **Variables de Entrada**:.[1]
- **Entrenamiento**: El modelo se entrena periódicamente con una muestra global de jugadores legítimos. Las nuevas entradas se puntúan según su "distancia de aislamiento".[22]
- **Interpretación**: Un puntaje de anomalía cercano a -1 indica una alta probabilidad de que el perfil sea un outlier (tramposo), mientras que valores cercanos a 1 indican normalidad.[28]

Cálculo del Score Final y Niveles de Riesgo

La probabilidad final se calcula mediante una suma ponderada de los tres enfoques anteriores,

ajustada por señales externas de confianza.[1]

Fórmula de Puntuación Ponderada

La probabilidad P se define como:

$$P = w_1(R) + w_2(S) + w_3(ML) - C$$

Donde:

- R: Score del sistema de reglas (0-1).
- S: Score estadístico basado en Z-scores normalizados (0-1).
- ML: Score de anomalía del Isolation Forest (escalado a 0-1).
- C: Factor de confianza basado en la antigüedad de la cuenta e inventario (skins caras reducen ligeramente la probabilidad de trampas flagrantes, aunque no las descartan).[1]

Clasificación de Salida

Nivel de Riesgo	Probabilidad	Descripción de Comportamiento
🟢 **Bajo**	0% - 25%	Comportamiento consistente con su rango y experiencia.
🟡 **Sospechoso**	26% - 60%	Anomalías leves; posible smurf o jugador con rachas de suerte.
🔴 **Alto Riesgo**	61% - 85%	Desviaciones estadísticas significativas; patrones de puntería no naturales.
☠️ **Rage Cheater**	86% - 100%	Valores imposibles para un ser

		humano; fraude casi seguro.

Arquitectura del Sistema Backend y Pipeline de Datos

Para soportar una aplicación web de alto rendimiento y baja latencia, se propone un stack tecnológico basado en Python y microservicios.[30]

Stack Tecnológico Recomendado

- **Lenguaje**: Python 3.11+ por su ecosistema de ciencia de datos (Pandas, Scikit-learn) y rendimiento asíncrono.
- **Framework API**: **FastAPI**. Permite una validación de datos estricta mediante Pydantic y un manejo eficiente de múltiples llamadas de red asíncronas.[30]
- **Base de Datos**:
  - **PostgreSQL**: Para el almacenamiento persistente de perfiles analizados y metadatos de partidas.
  - **Redis**: Como capa de caché para evitar consultas redundantes a las APIs externas y gestionar colas de tareas.[30]
- **Procesamiento en Segundo Plano**: **Celery**. La recolección de 100 partidas es una tarea lenta que no debe bloquear la respuesta HTTP. El usuario recibe un ID de tarea y el frontend consulta el progreso.[30]

Diseño del Pipeline de Análisis

1. **Entrada**: El usuario envía un SteamID64 al endpoint /analyze/{steamId}.
2. **Validación**: Se verifica el formato y se consulta en Redis si existe un análisis reciente (< 24h).
3. **Ingesta**: Se disparan tareas asíncronas para consultar Steam, Faceit y CS2.space en paralelo.[30]
4. **Feature Extraction**: Los resultados JSON se aplanan y se calculan las métricas de varianza y consistencia.
5. **Inferencia**: El vector de características se pasa por el modelo de Isolation Forest cargado en memoria.
6. **Puntuación**: Se aplican las reglas heurísticas y se genera el score final.
7. **Notificación**: Se actualiza el estado en la DB y el frontend muestra el resultado mediante WebSockets o polling.

Diseño de la Interfaz de Usuario (Frontend)

La interfaz debe ser informativa y visual, proporcionando "pruebas" de por qué un usuario ha

sido marcado como sospechoso para evitar acusaciones infundadas.

Visualización de Datos Clave

- **Dashboard de Riesgo**: Un indicador circular con el color del nivel de riesgo y el porcentaje de probabilidad.[1]
- **Gráfica de Evolución de K/D y HS%**: Un gráfico de líneas que muestra las últimas partidas. Se deben resaltar los puntos que son considerados outliers estadísticos mediante marcadores de color rojo.[15]
- **Desglose de Señales de Alerta**: Una lista de etiquetas (ej. "Reaction Time Outlier", "New Account", "High Win Streak") que explican los factores que más pesaron en el score final.[1]
- **Comparativa Pro**: Una tabla que compare las estadísticas del usuario frente a promedios de jugadores profesionales como s1mple o ZywOo. Si el usuario supera a los profesionales en métricas de puntería pura pero no en impacto, la sospecha aumenta.[21]

Manejo de Casos de Borde y Falsos Positivos

Un sistema anti-cheat probabilístico debe ser extremadamente cuidadoso con los falsos positivos, especialmente cuando se trata de jugadores legítimos de alto nivel o "smurfs".[2]

Smurfs vs Cheaters

Un smurf es un jugador de élite jugando en una cuenta nueva. Sus estadísticas serán altas, pero su comportamiento táctico será humano.

- **Diferenciador**: El smurf suele tener un "Positioning Rating" y "Utility Usage" muy altos, mientras que muchos cheaters dependen únicamente de la fuerza bruta de su puntería.[13]
- **Historial de Amigos**: El sistema analiza si el usuario tiene amigos con bans de VAC. Los cheaters suelen asociarse entre ellos; los smurfs suelen estar vinculados a otras cuentas de alto nivel.[5]

Jugadores Profesionales y Whitelisting

Para evitar el ridículo de marcar a una estrella de los e-sports como cheater, se mantiene una lista blanca (whitelist) dinámica de jugadores profesionales verificados. Si un SteamID está en la lista, el sistema simplemente muestra "Verified Pro" y bloquea el cálculo de probabilidad de trampa.[21]

Simulación de un Ejemplo Real

Input: SteamID 7656119842... (Cuenta detectada en un lobby de rango 22k Premier).[24]

Datos Obtenidos:

- Steam: Creada hace 14 meses. 410 horas en CS2. No bans previos.[5]
- Faceit: Nivel 4. K/D de 0.92 en Faceit (Rendimiento mediocre).[10]

- Partidas Premier: Últimas 20 partidas con K/D promedio de 2.1 y HS% de 72%. Winrate del 90%.[1]

Características Calculadas:

- **Z-score (HS%)**: +3.8 (Extremadamente alto para el rango).[25]
- **Skill Jump**: Pasó de 12,000 a 22,000 de Rating en 10 días.[2]
- **TTD Median**: 172ms (Biológicamente sospechoso).[16]
- **Consistency Score**: Desviación de K/D de solo 0.15 (Demasiado consistente).[15]

Resultado Final:

- Reglas: Activación de HS_EXTREME y TTD_IMPOSSIBLE.
- ML: Isolation Forest marca el perfil como "Strong Anomaly".
- **Score**: **94% - ☠️ Rage Cheater**.[1]

Escalabilidad, Mantenimiento y Monetización

Para transformar este diseño en un producto viable, se deben considerar aspectos de negocio y crecimiento.

- **Mejora sobre Valve**: A diferencia de VAC, que busca archivos en el PC, este sistema analiza el *output* del jugador. Es inmune a trampas externas o hardware DMA (Direct Memory Access) porque se basa en que los resultados finales siguen siendo estadísticamente imposibles.[1]
- **Escalabilidad**: El uso de una arquitectura orientada a eventos con Celery y Redis permite procesar miles de usuarios simultáneamente. El almacenamiento de las demos analizadas permite re-entrenar el modelo de ML periódicamente, mejorando la precisión a medida que el dataset crece.[30]
- **Monetización**:
  - **Modelo Freemium**: Búsquedas limitadas gratuitas; reportes detallados y seguimiento de "watchlist" mediante suscripción.
  - **API para Organizadores**: Venta de acceso a la API para plataformas de torneos que deseen realizar un "veteo" automático de jugadores antes de la inscripción.[9]
  - **Publicidad**: Integración en sitios de estadísticas ya existentes como un "Trust Badge" para perfiles limpios.[1]

Este diseño técnico proporciona una hoja de ruta clara para la implementación de un sistema de detección de trampas basado en datos, equilibrando la profundidad analítica con la viabilidad arquitectónica en un entorno de producción real.

Obras citadas

1. CS2 Tracker - Track CS2 Player Stats & AI Cheating Detection - Skinflow, fecha de

acceso: abril 9, 2026, https://skinflow.gg/cs2-tracker
2. I analyzed CS2 data from the past two years, and here's what I found regarding VAC-Live, cheaters and skill regression. - Reddit, fecha de acceso: abril 9, 2026, https://www.reddit.com/r/cs2/comments/1s7nfjb/i_analyzed_cs2_data_from_the_past_two_years_and/
3. Better Steam Web API Documentation, fecha de acceso: abril 9, 2026, https://steamwebapi.azurewebsites.net/
4. CS2 Api? : r/GlobalOffensive - Reddit, fecha de acceso: abril 9, 2026, https://www.reddit.com/r/GlobalOffensive/comments/1ggs07q/cs2_api/
5. Steam Web API - Valve Developer Community, fecha de acceso: abril 9, 2026, https://developer.valvesoftware.com/wiki/Steam_Web_API
6. steam-api | Skills Marketplace · LobeHub, fecha de acceso: abril 9, 2026, https://lobehub.com/skills/gal-criticism-gal-skills-steam-api
7. Steam Web API Documentation, fecha de acceso: abril 9, 2026, https://steamapi.xpaw.me/
8. Developer API chevron_right - CSFloat - Buy & Sell CS2 Skins on the Most Advanced Marketplace, fecha de acceso: abril 9, 2026, https://csfloat.com/support/wizard/4
9. Intro - FACEIT for Developers, fecha de acceso: abril 9, 2026, https://docs.faceit.com/docs/
10. Retrieving FACEIT Data - FACEIT for Developers, fecha de acceso: abril 9, 2026, https://docs.faceit.com/getting-started/Guides/retreiving-faceit-data/
11. Docs | FACEIT for Developers, fecha de acceso: abril 9, 2026, https://docs.faceit.com/docs/data-api/data/
12. Faceit Analyser API | faceit_analyser, fecha de acceso: abril 9, 2026, https://docs.faceitanalyser.com/
13. We are Leetify, ask us anything! : r/GlobalOffensive - Reddit, fecha de acceso: abril 9, 2026, https://www.reddit.com/r/GlobalOffensive/comments/n4v2so/we_are_leetify_ask_us_anything/
14. This game is full of cheaters not because it is but because most people don't know what they are talking about : r/cs2 - Reddit, fecha de acceso: abril 9, 2026, https://www.reddit.com/r/cs2/comments/1r42nbe/this_game_is_full_of_cheaters_not_because_it_is/
15. Use leetify to verify cheaters : r/cs2 - Reddit, fecha de acceso: abril 9, 2026, https://www.reddit.com/r/cs2/comments/1cw6lab/use_leetify_to_verify_cheaters/
16. I did not realize how many cheaters there were until I recently started looking at peoples stats in the games I'm playing with.. It's wild : r/cs2 - Reddit, fecha de acceso: abril 9, 2026, https://www.reddit.com/r/cs2/comments/1px3fub/i_did_not_realize_how_many_cheaters_there_were/
17. Counter-Strike: Global Offensive Access Match History - Valve ..., fecha de acceso: abril 9, 2026, https://developer.valvesoftware.com/wiki/Counter-Strike:_Global_Offensive_Access_Match_History

18. CS2 Stats - The Ultimate CS2 Stat Tracker - csstats.gg, fecha de acceso: abril 9, 2026, https://csstats.gg/
19. CS2 Stats, Detailed Analytics & Match Tracker, fecha de acceso: abril 9, 2026, https://scope.gg/
20. CS2 Stats Tracker — Check K/D, Rank & Stats Free | SteamAnalyst, fecha de acceso: abril 9, 2026, https://www.steamanalyst.com/tools/cs2-stats
21. CS2 Pro Player Stats and Dashboards: A Deep Dive into the Performance of Top eSports Icons - scope.gg, fecha de acceso: abril 9, 2026, https://scope.gg/guides/cs2-pro-player-stats-en/
22. Tips for Advanced Feature Engineering - Maarten Grootendorst, fecha de acceso: abril 9, 2026, https://www.maartengrootendorst.com/blog/feature-engineering/
23. Scope Meets Screen: Lessons Learned in Designing Composite Visualizations for Marksmanship Training Across Skill Levels - arXiv, fecha de acceso: abril 9, 2026, https://arxiv.org/html/2507.00333v1
24. Blatant Cheater Stack, High Elo Loss : r/cs2 - Reddit, fecha de acceso: abril 9, 2026, https://www.reddit.com/r/cs2/comments/1o193vq/blatant_cheater_stack_high_elo_loss/
25. Isolation Forest Algorithm: Anomaly Detection with Machine Learning, fecha de acceso: abril 9, 2026, https://deepfa.ir/en/blog/isolation-forest-algorithm-anomaly-detection-machine-learning
26. Anomaly Detection with Isolation Forest | by Kyle Hammerberg - Medium, fecha de acceso: abril 9, 2026, https://medium.com/@khammerberg53/anomaly-detection-with-isolation-forest-47c4996d2894
27. CS2 Player Comparison – Compare Skills, Stats & Winrates Side-by-Side - scope.gg, fecha de acceso: abril 9, 2026, https://scope.gg/compare/
28. IsolationForest — scikit-learn 1.8.0 documentation, fecha de acceso: abril 9, 2026, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html
29. Anomaly detection for feature engineering - V2 - Kaggle, fecha de acceso: abril 9, 2026, https://www.kaggle.com/code/danofer/anomaly-detection-for-feature-engineering-v2
30. FastAPI and Redis Tutorial: Build a High-Performance Python API, fecha de acceso: abril 9, 2026, https://redis.io/tutorials/develop/python/fastapi/
31. The simplest way to make FastAPI Redis work like it should - Hoop.dev, fecha de acceso: abril 9, 2026, https://hoop.dev/blog/the-simplest-way-to-make-fastapi-redis-work-like-it-should
32. Redis Anti-Patterns: Common Mistakes Every Developer Should Avoid, fecha de acceso: abril 9, 2026, https://redis.io/tutorials/redis-anti-patterns-every-developer-should-avoid/
33. CS2 Players Skill Comparing | Online Tool | SCOPE.GG, fecha de acceso: abril 9,

2026, https://scope.gg/comparing
34. CS2 Stat Tracking Guide (Using Leetify) - Steam Community, fecha de acceso: abril 9, 2026, https://steamcommunity.com/sharedfiles/filedetails/?id=3653428923
35. (PDF) A hierarchical approach for evaluating athlete performance with an application in elite basketball - ResearchGate, fecha de acceso: abril 9, 2026, https://www.researchgate.net/publication/377532856_A_hierarchical_approach_for_evaluating_athlete_performance_with_an_application_in_elite_basketball